
RECURSO · FACUNDO CORENGIA · BASDONAX

El Agente de IA que Atiende Clientes Reales

El código entero del agente que le montás a una empresa: memoria de verdad, mil conversaciones sin mezclarse y corriendo 24/7 en un servidor. Abierto y gratis, el mismo que armo en cámara.

GUÍA PRÁCTICA · PARA APLICAR, NO PARA MIRAR

EMPEZÁ POR ACÁ

Qué es lo que te estás llevando

Esto no es un resumen ni una lista de enlaces. Es **el repositorio completo** del agente de IA que construí en cámara: el mismo código, los mismos archivos y las mismas decisiones. Nada recortado, nada de "la versión buena la tenés adentro del curso".

Y la diferencia con los agentes de tutorial es una sola: **este está armado para atender gente de verdad**. No es una demo que anda mientras grabás y se cae cuando le escriben cinco personas juntas. Cada decisión rara que vas a ver adentro está tomada pensando en el día que ese agente esté atendiendo a los clientes de una empresa que te está pagando.

Podés arrancarlo hoy en tu máquina para entenderlo, y el mismo código sin tocar una línea se va a un servidor, se conecta a Telegram y —en el próximo video de la serie— a WhatsApp. **No hay una versión de práctica y otra de verdad: es el mismo agente.**

Acá está todo, público y con licencia MIT: usalo, copialo y vendé lo que armes arriba. No me tenés que pedir permiso.

github.com/fcori47/basdonax-ai-agentkit →

LA DIFERENCIA

Qué tiene esto que no tiene un agente de tutorial

Todo lo que sigue está resuelto adentro. Son las cosas que aparecen recién cuando el bot deja de ser tuyo y pasa a atender a los clientes de otro.

Mil personas, mil conversaciones que no se mezclan

RESUELTO

Cada conversación tiene su identificador, y ese identificador es después el número de WhatsApp o el chat de Telegram de cada persona. **El mismo agente atiende a todos sin cruzar una sola charla.** Es lo primero que rompe en los agentes de demo y lo primero que te va a reclamar un cliente.

La memoria sobrevive a las actualizaciones

RESUELTO

Modo prueba guarda en un archivo; **modo producción guarda en Postgres**, que aguanta varios procesos atendiendo a la vez y no se borra cuando subís una versión nueva. Se cambia con una variable: el agente no se entera de dónde vive su memoria.

Contesta como una persona, no como un formulario

RESUELTO

La respuesta sale **partida en varios mensajitos** en vez de un ladrillo de texto, y mientras piensa muestra "escribiendo...". Parece un detalle estético: es la diferencia entre que el cliente final sienta que le contesta alguien o que le contesta una máquina.

Un error con una persona no te voltea el bot

RESUELTO

Si una conversación explota, el resto sigue atendido. Y el error del proveedor **nunca se esconde**: llega tal cual a la pantalla, porque a las tres de la mañana lo que necesitás es saber qué pasó, no un "algo salió mal".

No contesta dos veces lo mismo

RESUELTO

Las plataformas de mensajería reenvían el mensaje cuando dudan si te llegó. Sin esto, el cliente recibe la misma respuesta dos y tres veces — y ahí ya perdiste la confianza del que te contrató.

El gancho para apagarlo y pasárselo a una persona

PREPARADO

El lugar exacto donde se decide **si contesta el bot o contesta un humano** ya está en el núcleo, esperando. Es *el* diferencial cuando vendés esto a una empresa: nadie quiere un robot que no se puede apagar en la mitad de una venta. La implementación completa entra con el video de WhatsApp.

MANOS A LA OBRA

Ponerlo a andar en 3 pasos

Empezás en tu máquina para entenderlo. Si tenés Python instalado, esto son cinco minutos: copió y pegá.

1 · BAJARLO E INSTALARLO

```
git clone https://github.com/fcori47/basdonax-ai-agentkit
cd basdonax-ai-agentkit
python -m venv .venv
```

Activás el entorno — en Windows: `.venv\Scripts\activate` · en Mac o Linux: `source .venv/bin/activate`

Y después: `pip install -r requirements.txt`

2 · PONER UNA SOLA CLAVE

Copiás el archivo de ejemplo (`copy .env.example .env` en Windows, `cp` en Mac) y completás **un solo** proveedor:

Claude → `console.anthropic.com` · **OpenAI** → `platform.openai.com` · **Gemini** → `aistudio.google.com`

Con uno alcanza. Si cargás varios, los intercambiás desde la web con un botón — **el mismo agente, el proveedor que le convenga a cada cliente.**

3 · HABLARLE

`python servidor.py` → abris **localhost:8000** y ya tenés la plataforma de pruebas: le hablás, le cambiás el modelo en caliente, le editás la personalidad y **ves cuántos tokens gastó cada mensaje**.

¿Preferís la terminal? `python chat.py`

¿Lo querés en Telegram? `python bot_telegram.py`

Esa pantalla de tokens es la que te dice si el proyecto le cierra al cliente antes de prometerle nada. Y el archivo .env, donde viven las claves, nunca se sube: ya está ignorado en el repositorio.

EL MAPA

Qué hay adentro y para qué sirve cada cosa

Son pocos archivos a propósito. Si vas a abrir uno solo, que sea el primero.

agente.py

EMPEZÁ POR ACÁ

El agente entero. El grafo son doce líneas; todo lo demás alrededor es explicación y ayudantes. Recibe texto y devuelve texto: no sabe si lo llamaron desde la terminal, la web, Telegram o WhatsApp. Esa frontera es lo que te deja enchufarle canales nuevos sin volver a tocarlo nunca.

herramientas.py

LO QUE MÁS VAS A TOCAR

Lo que el agente sabe hacer además de hablar — o sea, **lo que realmente le vas a cobrar a una empresa**: consultar su stock, anotar un turno, buscar en su sistema. Hoy trae una de ejemplo, el clima. Agregar la tuya es escribir una función y sumarla a una lista: el grafo ya está armado para usarlas.

memoria.py

Dónde se guardan las conversaciones. Tres opciones intercambiables: en memoria (se borra al cerrar), un archivo para desarrollar y **Postgres para producción**. Cambiar dónde viven las charlas no toca el agente.

canales/telegram.py

El bot de Telegram, y **el molde de cómo se conecta cualquier canal**. Mensaje que entra → el agente → mensaje que sale. WhatsApp va a ser un archivo hermano de este, con la misma forma.

prompts/sistema.md

La personalidad, en un archivo de texto plano. Se lee **en cada mensaje**, no una vez al arrancar: lo editás, guardás, y el mensaje siguiente ya sale distinto, sin reiniciar nada. Con el agente andando en producción, ajustarle el tono a un cliente son treinta segundos.

tests/ y el Dockerfile

Más de 60 pruebas automáticas que **no gastan un solo token** (usan un modelo falso), para que puedas cambiar cosas sin miedo a romper lo que ya está atendiendo. Y un Dockerfile que empaqueta el bot para dejarlo corriendo en un servidor.

LOS VIDEOS

Qué se construye en cada uno

El repositorio es el resultado de esta serie. Cada video agrega una capa arriba del mismo código: no se rehace nada, se le suma. Ese es justamente el punto.

1 • El agente, desde cero

PUBLICADO

Se arma el agente entero con Claude Code: el grafo, la elección del modelo, la plataforma de pruebas y —lo más importante— **la memoria**. Es el video donde queda claro por qué las conversaciones no se mezclan, que es lo que después lo hace servible para una empresa.

Cómo CREAR un agente de IA con Claude Code desde cero →

2 • Tu propio servidor de IA

PUBLICADO

Un servidor propio con Coolify montado de cero. Es el paso que separa "anda mientras tengo la compu prendida" de "**le anda al cliente un domingo a las tres de la mañana**", y el que después te sirve para todo lo demás que quieras dejar corriendo.

Cómo INSTALAR tu propio SERVIDOR de IA desde cero →

3 • El agente en Telegram

PUBLICADO

El mismo agente contestando en el teléfono, y después subido al servidor para que atienda solo. Acá se ve el truco de todo esto: **el chat de cada persona es su propia conversación**, y el mismo bot atiende a mil sin mezclar una sola.

Cómo CONECTAR un agente de IA a Telegram →

4 • WhatsApp

EN CAMINO

El que estabas esperando, y el que se vende. Ahí entra todo lo que separa un bot de demo de uno que atiende clientes: la firma de Meta, juntar la ráfaga de tres mensajes seguidos, la ventana de 24 horas y **el traspaso a una persona**. En el repositorio ya está escrito qué pieza va dónde para que entre sin reescribir nada.

El dato que da vuelta la discusión de n8n contra código

n8n usa LangChain por dentro. El nodo del agente se llama literalmente `@n8n/n8n-nodes-langchain.agent` y está en el JSON de cualquier flujo tuyo: lo comprobás en diez segundos. No son dos mundos enfrentados, es la misma base con o sin la capa del medio. Lo que cambió no es la herramienta: es que el precio de entrada del código —saber programar— desapareció.

LO QUE CASI NADIE EXPLICA

Tres cosas que vas a entender leyendo esto

Y que valen más que el código en sí, porque te van a aparecer en cualquier proyecto que le entregues a un cliente.

Las APIs de los modelos no recuerdan nada

Cada llamada es independiente. Que el agente "se acuerde" de lo que hablaron es **puro trabajo tuyo**: le volvé a mandar la conversación entera cada vez. Acá eso lo resuelve un identificador de conversación, y ese identificador es exactamente el que después va a ser el número de WhatsApp de cada cliente de la empresa. Ahí está todo el truco.

La memoria es lo que te come el margen

Como la conversación viaja entera en cada mensaje, decirle que recuerde 20 mensajes en vez de 10 es, más o menos, el doble de gasto en charlas largas. En una empresa con cientos de conversaciones por día eso es **la diferencia entre un proyecto que cierra y uno que te deja pagando la cuenta**. Es un número que ajustás sin reiniciar nada.

El docstring de una herramienta es el prompt

Esa descripción de una línea que ponés arriba de la función **no es un comentario para programadores: es lo único que lee el modelo** para decidir si la herramienta le sirve y qué mandarle. Si está mal escrita, la herramienta no se usa nunca y te vas a volver loco buscando el error en el código. No está en el código.

REGALO APARTE

Los problemas que ya te dejé resueltos

Cada uno de estos me costó horas, y todos aparecen recién cuando el agente sale a producción. Están arreglados en el código y explicados en el AGENTS.md, así que no los vas a sufrir vos.



La conversación larga que de golpe devuelve un error sin explicación. Cuando el agente usa una herramienta, esa vuelta son tres mensajes atados que el proveedor exige juntos. Si el recorte de memoria cae justo en el medio, te rechaza el pedido sin decirte por qué —y recién cuando la charla se hizo larga, o sea con el cliente adentro. Acá el recorte corta por turnos completos y nunca los parte.

- ✓ **Postgres: la versión del paquete no es un capricho.** La serie 2.x se lleva por delante el resto del proyecto, se instala igual y te lo avisa en un renglón perdido entre otros veinte. Tiene que ser 3.x, y en Windows el otro paquete va con corchetes o falla aunque figure instalado.
- ✓ **El bot subido al servidor que aparece "caído" y está perfecto.** Como sale él a buscar los mensajes, no expone puerto: el panel le asigna un dominio solo, nadie contesta ahí y lo marca como enfermo. No está roto — hay que borrarle el dominio.
- ✓ **El bot que contesta a veces sí y a veces no.** La falla más confusa de todas: si te queda corriendo en el servidor y en tu computadora, los dos piden los mismos mensajes y se los reparten al azar. Mitad de las respuestas salen de cada lado.
- ✓ **Las conversaciones que desaparecen en cada actualización.** Adentro de un contenedor, el archivo de la base vive en un disco que se descarta en cada despliegue. En un servidor va Postgres, y ahí la memoria de cada persona sobrevive a que vos subas una versión nueva.
- ✓ **Pedirle herramientas a un modelo que razona por el camino viejo.** Toda la familia nueva de OpenAI lo rechaza; la única otra salida que te ofrece el error es apagarle el razonamiento al modelo, que es pagar por uno y usar otro. Ya está apuntado al camino correcto.

HONESTIDAD

Lo que todavía no tiene

Te lo digo yo antes de que lo descubras: **no tiene WhatsApp, no tiene base de conocimiento y tiene una sola herramienta de ejemplo.** No es un olvido — cada una de esas cosas es un video de la serie, y prefiero que las armemos juntos en pantalla antes que dejarte una caja negra que funciona y no entendés. El día que se rompa con un cliente adentro, esa diferencia es todo.

Lo que sí está hecho es **que ninguna de esas cosas te obligue a reescribir el agente cuando llegue:** la forma de un canal, el gancho para pasarle la conversación a una persona y la respuesta partida en varios mensajes ya están en el núcleo. Cuando salga el video de WhatsApp, es un archivo nuevo. El resto queda igual.

Consejo práctico: abrí Claude Code (o Codex, o Cursor) dentro de la carpeta y pedile que te agregue la herramienta que necesita tu cliente. Lee solo el AGENTS.md, ya sabe cómo está armado y cuáles son las trampas. Eso es lo que a mí me ahorra la mayor parte del trabajo.

EL SIGUIENTE PASO

Tener el código es la parte fácil

Y ya la tenés resuelta, gratis. Lo que separa a alguien con un agente andando de alguien que se lo entrega a una empresa es todo lo otro: a quién le sirve, qué le resolvés en concreto y cómo se lo ofrecés.



Empezá por el primero de la serie

El video donde se construye desde cero todo esto que te acabás de bajar. Si vas a abrir el código, mirá este antes: te ahorra leer la mitad.

► **Ver el video**

Si estás armando el tuyo y querés que veamos tu caso concreto —qué le vas a resolver a quién— escribime directo y me contás en qué andás. Te respondo yo, no un robot.

Hablame por WhatsApp →

facundocorengia.com

FACUNDO CORENGIA · BASDONAX